Q1 A data engineer is configuring an AWS Glue job to read data from an Amazon S3 bucket. The data engineer has set up the necessary AWS Glue connection details and an associated IAM role. However, when the data engineer attempts to run the AWS Glue job, the data engineer receives an error message that indicates that there are problems with the Amazon S3 VPC gateway endpoint.

The data engineer must resolve the error and connect the AWS Glue job to the S3 bucket.

Which solution will meet this requirement?

Options

A. Update the AWS Glue security group to allow inbound traffic from the Amazon S3 VPC gateway endpoint.

B. Configure an S3 bucket policy to explicitly grant the AWS Glue job permissions to access the S3 bucket.

C. Review the AWS Glue job code to ensure that the AWS Glue connection details include a fully qualified domain name.

D. Verify that the VPC's route table includes inbound and outbound routes for the Amazon S3 VPC gateway endpoint.

Correct Answer

D. Verify route table entries for S3 VPC gateway endpoint

Explanation

- Problem:
  - Glue job cannot access S3 via **VPC endpoint**
  - Error points to **S3 gateway endpoint issue**

Why Option D is correct:

- **S3 Gateway Endpoint** requires:
  - Proper **route table entries**
  - Route like:
    pl-xxxx (S3 prefix list) → VPC endpoint
- If route is missing:
  Traffic cannot reach S3

Why other options are incorrect:

- **A** → Gateway endpoints do NOT use security groups
- **B** → IAM already configured; issue is networking
- **C** → Not related to VPC endpoint connectivity

Key Exam Tip

- **S3 Gateway Endpoint = Route Table (NOT Security Group)**

Q2 A retail company has a customer data hub in an Amazon S3 bucket. Employees from many countries use the data hub to support company-wide analytics. A governance team must ensure that the company's data analysts can access data only for customers who are within the same country as the analysts.

Which solution will meet these requirements with the LEAST operational effort?

Options

A. Create a separate table for each country's customer data. Provide access to each analyst based on the country that the analyst serves.

B. Register the S3 bucket as a data lake location in AWS Lake Formation. Use the Lake Formation row-level security features to enforce the company's access policies.

C. Move the data to AWS Regions that are close to the countries where the customers are. Provide access to each analyst based on the country that the analyst serves.

D. Load the data into Amazon Redshift. Create a view for each country. Create separate IAM roles for each country to provide access to data from each country. Assign the appropriate roles to the analysts.

Correct Answer

B. Use AWS Lake Formation with row-level security

Explanation

- Requirement:

- Fine-grained access → **country-level filtering**
- Minimal operational effort
- Data already in **S3 (data lake)**

Why Option B is correct:

- **AWS Lake Formation**:
  - Centralized data lake governance
  - Supports **row-level security (RLS)**
  - Enforces access policies dynamically
- No need to:
  - Duplicate data
  - Create multiple tables/views

Efficient + scalable + least maintenance

Why other options are incorrect:

- **A** → Data duplication + high maintenance
- **C** → Region-based movement doesn't enforce access control
- **D** → Redshift adds complexity and operational overhead

Key Exam Tip

- **Fine-grained access on S3 data lake = Lake Formation (Row-level security)**

Q3 A media company wants to improve a system that recommends media content to customers based on user behavior and preferences. To improve the recommendation system, the company needs to incorporate insights from third-party datasets into the company's existing analytics platform.

The company wants to minimize the effort and time required to incorporate third-party datasets.

Which solution will meet these requirements with the LEAST operational overhead?

Options

A. Use API calls to access and integrate third-party datasets from AWS Data Exchange.

B. Use API calls to access and integrate third-party datasets from AWS DataSync.

C. Use Amazon Kinesis Data Streams to access and integrate third-party datasets from AWS CodeCommit repositories.

D. Use Amazon Kinesis Data Streams to access and integrate third-party datasets from Amazon Elastic Container Registry (Amazon ECR).

Correct Answer

A. Use AWS Data Exchange APIs

Explanation

- Requirement:
  - Integrate **third-party datasets**
  - Minimal effort & fastest setup

Why Option A is correct:

- **AWS Data Exchange**:
  - Purpose-built for **third-party data subscriptions**
  - Provides datasets via:
    - S3
    - APIs
  - No need to build ingestion pipelines

Plug-and-play data integration → least operational overhead

Why other options are incorrect:

- **B (DataSync)** → Used for data transfer (on-prem ↔ AWS), not third-party data marketplace
- **C (Kinesis + CodeCommit)** → Code repositories ≠ data sources
- **D (Kinesis + ECR)** → ECR stores container images, not datasets

Overview of Services in Options

AWS Data Exchange (Correct Answer)

- Marketplace for **third-party datasets**
- Subscribe → access data instantly
- Supports:
  - S3 delivery
  - API-based access
- Use case:
  Financial data, weather data, analytics datasets

AWS DataSync

- Transfers data between:
  - On-prem → AWS
  - AWS → AWS
- Optimized for:
  - Large-scale migration
  - File/object transfer
- Not for discovering or subscribing to datasets

Amazon Kinesis Data Streams

- Real-time **streaming data ingestion**
- Handles:
  - Logs
  - Clickstreams
  - IoT data
- Not a data source marketplace

AWS CodeCommit

- Managed **Git repository service**
- Stores source code

- Not for datasets

Amazon ECR (Elastic Container Registry)

- Stores **Docker container images**
- Used in:
    - ECS / EKS deployments
- Not for data ingestion

Key Exam Tip

- **Third-party datasets → AWS Data Exchange**
- **Data transfer → DataSync**
- **Streaming → Kinesis**

Q4 A financial company wants to implement a data mesh. The data mesh must support centralized data governance, data analysis, and data access control. The company has decided to use AWS Glue for data catalogs and ETL operations.

Which combination of AWS services will implement a data mesh? *(Choose two.)*

Options

A. Use Amazon Aurora for data storage. Use an Amazon Redshift provisioned cluster for data analysis.

B. Use Amazon S3 for data storage. Use Amazon Athena for data analysis.

C. Use AWS Glue DataBrew for centralized data governance and access control.

D. Use Amazon RDS for data storage. Use Amazon EMR for data analysis.

E. Use AWS Lake Formation for centralized data governance and access control.

Correct Answers

B. Amazon S3 + Athena
E. AWS Lake Formation

Explanation

- Data mesh requires:
    - **Storage** → S3 (data lake)
    - **Query engine** → Athena (serverless analytics)
    - **Governance** → Lake Formation

These together create a scalable, governed data mesh architecture

Why other options are incorrect:

- **A & D** → Traditional DB approach, not data mesh
- **C** → DataBrew is for data preparation, not governance

Key Exam Tip

Data Mesh = S3 + Athena + Lake Formation

Q5 A data engineer maintains custom Python scripts used by multiple AWS Lambda functions. Updating scripts requires manual updates across all functions.

Which solution will reduce manual effort?

Options

A. Store pointer to scripts in S3 execution context

B. Package scripts into Lambda layers and apply to functions

C. Store pointer in environment variables

D. Use Lambda aliases

Correct Answer

B. Use Lambda Layers

Explanation

- **Lambda Layers**:
    - Centralized shared code

    - Update once → all functions benefit
    - Reduces duplication and manual effort

Why other options are incorrect:

- **A & C** → Indirect referencing, still manual
- **D** → Alias is for versioning, not shared code

Key Exam Tip

Shared code across Lambda = Layers

Q6

A company created an ETL pipeline in AWS Glue. The pipeline requires crawling, transformation, loading to S3, and orchestration.

Which service will meet these requirements MOST cost-effectively?

Options

A. AWS Step Functions

B. AWS Glue workflows

C. AWS Glue Studio

D. Amazon MWAA (Airflow)

Correct Answer

B. AWS Glue Workflows

Explanation

- Glue Workflows:
    - Native orchestration for Glue jobs + crawlers
    - No extra service cost

    - Fully integrated

Why other options are incorrect:

- **A** → Extra cost and complexity
- **C** → UI tool, not orchestration
- **D** → Expensive for simple workflows

Key Exam Tip

Glue orchestration = Glue Workflows

Q7

A financial services company stores data in Amazon Redshift. A web app needs to run real-time queries with minimal operational overhead.

Options

A. WebSocket connections

B. Amazon Redshift Data API

C. JDBC connections

D. S3 + S3 Select

Correct Answer

B. Amazon Redshift Data API

Explanation

- Redshift Data API:
    - No persistent connections
    - No drivers required
    - Uses HTTP calls → ideal for web apps

Why other options are incorrect:

- **A** → Not supported
- **C** → Requires connection management
- **D** → Not real-time querying from Redshift

Key Exam Tip

Serverless access to Redshift = Data API

Q8 A company uses Amazon Athena and needs to separate query access and permissions across teams.

Options

A. S3 bucket per use case

B. Athena Workgroups + IAM tags

C. IAM roles per use case

D. Glue Data Catalog policies

Correct Answer

B. Athena Workgroups + IAM policies with tags

Explanation

- **Athena Workgroups**:
    - Separate queries, limits, and costs
- **IAM + Tags**:
    - Fine-grained access control

Best combination for multi-team environments

Why other options are incorrect:

- **A** → Storage-level control only
- **C** → Not scalable
- **D** → Catalog control only, not query isolation

Key Exam Tip

Athena multi-tenant control = Workgroups + IAM

Q9 A data engineer needs to schedule a workflow that runs a set of AWS Glue jobs every day. The data engineer does not require the Glue jobs to run or finish at a specific time.

Which solution will run the Glue jobs in the MOST cost-effective way?

Options

A. Choose the FLEX execution class in the Glue job properties.

B. Use the Spot Instance type in Glue job properties.

C. Choose the STANDARD execution class in the Glue job properties.

D. Choose the latest version in the GlueVersion field in the Glue job properties.

Correct Answer

A. Choose the FLEX execution class

Explanation

- Requirement:
  - Daily jobs
  - No strict timing requirement
  - **Cost optimization is priority**

Why Option A is correct:

- **Glue FLEX execution class**:
  - Lower cost compared to STANDARD

- Jobs may start later (flexible scheduling)
- Ideal when timing is not critical

Best choice for **cost savings**

Why other options are incorrect:

- **B** → Glue does not directly use Spot instances
- **C** → STANDARD is faster but more expensive
- **D** → Version selection does not affect cost directly

Key Exam Tip

- **Glue FLEX = Cheapest (flexible start time)**
- **Glue STANDARD = Faster but expensive**

Q10

A data engineer needs to create an AWS Lambda function that converts the format of data from .csv to Apache Parquet. The Lambda function must run only if a user uploads a .csv file to an Amazon S3 bucket.

Which solution will meet these requirements with the LEAST operational overhead?

Options

A. Create an S3 event notification that has an event type of s3:ObjectCreated:*. Use a filter rule to generate notifications only when the suffix includes .csv. Set the Lambda function ARN as the destination.

B. Create an S3 event notification with event type s3:ObjectTagging:* for objects tagged as .csv.

C. Create an S3 event notification with event type s3:* and filter suffix .csv.

D. Create an S3 event notification → SNS → Lambda subscription

Correct Answer

A. S3 ObjectCreated event + suffix filter (.csv) → Lambda

Explanation

- Requirement:
  - Trigger Lambda only on **CSV uploads**
  - Minimal operational overhead

Why Option A is correct:

- Uses:
  - **S3 Event Notification (ObjectCreated)**
  - **Suffix filter = .csv**
  - Direct trigger to Lambda

Simple, event-driven, no extra services

Why other options are incorrect:

- **B** → Requires tagging → extra step
- **C** → Too broad (s3:*) → inefficient
- **D** → SNS adds unnecessary complexity

Key Exam Tip

- **S3 → Lambda trigger = ObjectCreated + filter (prefix/suffix)**

Q11 A data engineer needs Amazon Athena queries to finish faster. The data engineer notices that all the files the Athena queries use are currently stored in uncompressed .csv format. The data engineer also notices that users perform most queries by selecting a specific column.

Which solution will MOST speed up the Athena query performance?

Options

A. Change the data format from CSV to JSON format. Apply Snappy compression.

B. Compress the CSV files by using Snappy compression.

C. Change the data format from CSV to Apache Parquet. Apply Snappy compression.

D. Compress the CSV files by using gzip compression.

Correct Answer

C. Convert CSV → Parquet + apply Snappy compression

Explanation

- Key observations:
  - Data is in **CSV (row-based)**
  - Queries access **specific columns**

Why Option C is correct:

- **Parquet (columnar format)**:
  - Reads only required columns
  - Reduces data scanned → faster queries
- **Snappy compression**:
  - Fast compression/decompression
  - Reduces storage + improves performance

Best combination for Athena optimization

Why other options are incorrect:

- **A (JSON)** → Still row-based, inefficient
- **B & D (CSV compression)** → Improves size, NOT column access

Key Exam Tip

- **Athena optimization = Columnar format (Parquet/ORC)**
- **Compression = Snappy (best performance balance)**

Q12

A manufacturing company collects sensor data from its factory floor to monitor and enhance operational efficiency. The company uses Amazon Kinesis Data Streams to publish the data. Then Amazon Kinesis Data Firehose writes the data to an Amazon S3 bucket.

The company needs to display a real-time view of operational efficiency on a large screen in the manufacturing facility.

Which solution will meet these requirements with the LOWEST latency?

Options

A. Use Amazon Managed Service for Apache Flink (Kinesis Data Analytics) to process the sensor data. Use a connector to write data to Amazon Timestream. Use Timestream as a source for a Grafana dashboard.

B. Use S3 event notifications → Lambda → Aurora → QuickSight dashboard.

C. Use Flink → Firehose → Timestream → QuickSight dashboard.

D. Use AWS Glue bookmarks → Timestream → Grafana dashboard.

Correct Answer

A. Flink (Kinesis Data Analytics) → Timestream → Grafana

Explanation

- Requirement:
  - **Real-time dashboard**
  - **Lowest latency**
  - Streaming data from sensors

Why Option A is correct:

- **Amazon Managed Service for Apache Flink**:
  - Real-time stream processing (milliseconds latency)
- **Amazon Timestream**:
  - Time-series database optimized for IoT/sensor data
- **Grafana**:
  - Real-time visualization dashboards

End-to-end **streaming pipeline → lowest latency**

Why other options are incorrect:

- **B** → S3 + Lambda introduces delay (batch/event-based)
- **C** → Firehose adds buffering → higher latency
- **D** → Glue is batch-oriented, not real-time

Key Exam Tip

- **Real-time streaming analytics = Kinesis + Flink**
- **Time-series data = Timestream**
- **Real-time dashboards = Grafana**

Q13

A company stores daily records of the financial performance of investment portfolios in .csv format in an Amazon S3 bucket. A data engineer uses AWS Glue crawlers to crawl the S3 data.

The data engineer must make the S3 data accessible daily in the AWS Glue Data Catalog.

Which solution will meet these requirements?

Options

A. Create an IAM role with AmazonS3FullAccess. Associate with crawler. Set S3 path. Schedule daily. Configure output to a new S3 path.

B. Create an IAM role with AWSGlueServiceRole policy. Associate with crawler. Set S3 path as data store. Schedule crawler daily. Specify a database name for output.

C. Create IAM role with S3FullAccess. Allocate DPUs to crawler daily. Specify database name.

D. Create IAM role with AWSGlueServiceRole. Allocate DPUs. Configure output to S3 path.

Correct Answer

B. Use AWSGlueServiceRole + crawler + schedule + Glue Data Catalog database

Explanation

- Requirement:
    - Crawl S3 data daily

    - Update **Glue Data Catalog**
    - Minimal configuration

Why Option B is correct:

- **AWSGlueServiceRole**:
    - Default managed role for Glue crawlers
- **Crawler setup**:
    - S3 path as data source
    - Schedule (daily)
    - Output → Glue Data Catalog database

Fully managed + correct architecture

Why other options are incorrect:

- **A** → Output to S3 is incorrect (crawler outputs to catalog)
- **C & D** → Crawlers do NOT require DPUs configuration

Key Exam Tip

- **Glue Crawler → Updates Data Catalog (NOT S3)**
- **Use AWSGlueServiceRole (standard practice)**

Q14

A company loads transaction data daily into Amazon Redshift tables. The company wants to track which tables have been loaded and which still need to be loaded.

A data engineer stores load statuses in Amazon DynamoDB and creates an AWS Lambda function to publish these details.

How should the data engineer invoke the Lambda function to write load statuses to DynamoDB?

Options

A. Use a second Lambda triggered by CloudWatch events

B. Use Amazon Redshift Data API → EventBridge → Lambda

C. Use Amazon Redshift Data API → SQS → Lambda

D. Use a second Lambda triggered by CloudTrail events

Correct Answer

B. Redshift Data API → EventBridge → Lambda

Explanation

- Requirement:
  - Trigger Lambda when Redshift load events occur
  - Event-driven, scalable, low overhead

Why Option B is correct:

- **Redshift Data API**:
  - Executes queries and emits events
- **Amazon EventBridge**:
  - Captures events and routes them
- **Lambda**:
  - Processes and writes to DynamoDB

Clean **event-driven architecture**

Why other options are incorrect:

- **A & D** → Unnecessary extra Lambda layer
- **C** → SQS adds complexity, not needed

Key Exam Tip

- **Event-driven AWS pattern = EventBridge → Lambda**
- Prefer **EventBridge over SQS** when routing events

Q15

A data engineer needs to securely transfer **5 TB of data** from an on-premises data center to an Amazon S3 bucket. Approximately **5% of the data changes daily**, and updates must be regularly transferred. The process must be **automated and scheduled**.

Which AWS service should be used in the MOST operationally efficient way?

Options

A. AWS DataSync

B. AWS Glue

C. AWS Direct Connect

D. Amazon S3 Transfer Acceleration

Correct Answer

A. AWS DataSync

Explanation

- Requirements:
  - Large data transfer (TB scale)
  - Incremental updates (5% daily changes)
  - Automation + scheduling
  - Secure transfer

Why Option A is correct:

- **AWS DataSync**:
  - Optimized for **on-prem → S3 transfers**
  - Supports **incremental sync (only changed data)**
  - Built-in **scheduling & automation**
  - Handles encryption and performance optimization

Best for **bulk + recurring data transfer**

Why other options are incorrect:

- **B (Glue)** → ETL service, not for data transfer
- **C (Direct Connect)** → Network connection, not transfer automation
- **D (Transfer Acceleration)** → Speeds uploads but no scheduling/incremental sync

Key Exam Tip

- **Bulk + recurring data transfer = AWS DataSync**
- **Incremental sync requirement → Always think DataSync**

Q16

A company uses an on-premises Microsoft SQL Server database and migrates data monthly to Amazon RDS for SQL Server. The cost has increased, and the company needs a **cost-effective solution with minimal downtime**.

Which AWS service should be used?

Options

A. AWS Lambda

B. AWS Database Migration Service (AWS DMS)

C. AWS Direct Connect

D. AWS DataSync

Correct Answer

B. AWS Database Migration Service (AWS DMS)

Explanation

- Requirements:
    - Database migration
    - Minimal downtime
    - Cost-effective

Why Option B is correct:

- **AWS DMS**:

- Supports **continuous replication (CDC)**
- Enables **minimal downtime migration**
- Optimized for database-to-database migration

Ideal for **SQL Server → RDS migration**

Why other options are incorrect:

- **A** → Not for DB migration
- **C** → Network connection only
- **D** → File transfer, not DB migration

Key Exam Tip

- **Database migration + minimal downtime = AWS DMS**

Q17

A data engineer is building a pipeline using AWS Glue ETL jobs. The pipeline processes data from Amazon RDS and MongoDB, transforms it, and loads it into Amazon Redshift. Updates must occur **every hour**.

Which combination of tasks meets the requirements with the LEAST operational overhead? *(Choose two.)*

Options

A. Configure AWS Glue triggers to run ETL jobs every hour

B. Use AWS Glue DataBrew for data preparation

C. Use AWS Lambda to schedule ETL jobs

D. Use AWS Glue connections for connectivity

E. Use Redshift Data API for loading

Correct Answers

A. AWS Glue triggers
D. AWS Glue connections

Explanation

Why A is correct:

- **Glue Triggers**:
  - Native scheduling (cron/hourly)
  - No need for external services

Why D is correct:

- **Glue Connections**:
  - Simplifies connectivity to:
    - RDS
    - MongoDB
    - Redshift

Fully managed, minimal setup

Why other options are incorrect:

- **B** → DataBrew is for visual prep, not ETL pipelines
- **C** → Lambda adds unnecessary complexity
- **E** → Data API is not required for Glue-based loads

Key Exam Tip

- **Glue scheduling = Glue Triggers**
- **Connectivity = Glue Connections**

Q18

A company uses an Amazon Redshift cluster on RA3 nodes. The company wants to scale read and write capacity to meet demand. A data engineer needs to enable **concurrency scaling**.

Which solution will meet this requirement?

Options

A. Turn on concurrency scaling in WLM for Redshift Serverless workgroups

B. Turn on concurrency scaling at the WLM queue level in the Redshift cluster

C. Enable concurrency scaling during creation of a new Redshift cluster

D. Enable concurrency scaling using daily usage quota settings

Correct Answer

B. Enable concurrency scaling at the WLM queue level

Explanation

- Requirement:
  - Handle high concurrent queries
  - Scale automatically

Why Option B is correct:

- **Concurrency Scaling in Redshift**:
  - Enabled per **WLM (Workload Management) queue**
  - Automatically adds transient clusters
  - Handles spikes in query load

This is the **actual configuration point**

Why other options are incorrect:

- **A** → For Redshift Serverless, not RA3 clusters
- **C** → Not configured at cluster creation level
- **D** → Quotas control usage, not enable scaling

Key Exam Tip

- **Redshift scaling = WLM Queue setting**
- **Concurrency Scaling ≠ Cluster-level setting**

Q19 A data engineer must orchestrate a series of Amazon Athena queries that run every day. Each query can run for more than 15 minutes.

Which combination of steps will meet these requirements MOST cost-effectively? *(Choose two.)*

Options

A. Use AWS Lambda + Athena Boto3 start_query_execution

B. Use AWS Step Functions with Wait state + get_query_execution polling

C. Use AWS Glue Python shell job

D. Use Glue Python shell with sleep polling

E. Use Amazon MWAA (Airflow)

Correct Answers

A. Lambda + Athena API
B. Step Functions + Wait state orchestration

Explanation

Why A is correct:

- Lambda can:
  - Trigger Athena queries programmatically
  - Use start_query_execution API

Why B is correct:

- **Step Functions**:
  - Handles long-running workflows (>15 min)
  - Wait state avoids continuous polling cost
  - Orchestrates sequential execution

Best combination:

- Lambda → start query
- Step Functions → manage workflow

Why other options are incorrect:

- **C & D** → Glue not ideal for orchestration
- **E** → MWAA is expensive for simple workflows

Key Exam Tip

- **Athena orchestration = Lambda + Step Functions**
- **Long-running workflows → Step Functions (not Lambda alone)**

Q20 A company is migrating big data workloads (Pig, Spark, HBase, Flink) to AWS. The workloads process **petabytes of data** and require **high performance** with reduced operational overhead.

Which ETL service should be used?

Options

A. AWS Glue

B. Amazon EMR

C. AWS Lambda

D. Amazon Redshift

Correct Answer

B. Amazon EMR

Explanation

- Requirements:
    - Big data ecosystem (Spark, HBase, Flink, Pig)
    - Petabyte-scale processing
    - Maintain performance

Why Option B is correct:

- **Amazon EMR**:
    - Native support for:
        - Spark
        - Hadoop
        - HBase
        - Flink
    - Scales for big data workloads
    - Fully managed cluster

Best fit for **existing big data frameworks**

Why other options are incorrect:

- **A (Glue)** → Limited for heavy custom big data workloads
- **C (Lambda)** → Not for big data processing
- **D (Redshift)** → Data warehouse, not ETL engine

Key Exam Tip

- **Big Data frameworks (Spark/Hadoop) → EMR**
- **Serverless ETL → Glue (not for heavy legacy workloads)**